

Build a Claude Code Plugin

BEEF CO · PLATFORM ENGINEERING

A plugin is a folder of files that teaches Claude Code a workflow your team repeats. A great plugin isn't just a pile of skills. The useful ones turn skills, agents, and hooks into a repeatable workflow. You've seen **adoption-os** find the patterns in how your team uses Claude Code. Here's how to build one for a workflow of your own.

first plugin **Input** -> **Skill** does the work -> **File/output** -> README shows exactly how to run it

1 Pick one workflow

Do not build a general productivity plugin. Pick one repeated engineering task that currently happens by hand.

- Scaffold a new service from company conventions.
- Review a module against platform standards.
- Turn a failed CI run into likely cause and next action.
- Convert dependency alerts into a remediation plan.

The more narrow the scope, the easier to test.

2 Write the plugin sentence

Be concrete about who it's for, what it does, and what it replaces. Below is a template to help you draft your plugin prompt:

TEMPLATE *This plugin helps [person] do [task] using [input/context] so they can produce [specific output].*

GOOD EXAMPLE This plugin helps a platform engineer scaffold a new microservice using Beef Co engineering conventions so they can create the standard service files without copying from old repos or asking another team how it is done.

This is 90% of the work. Be specific, and give examples.

Tip: input the prompt you've written into Claude Code and ask it how it would improve it further using best practices.

3 Know the Parts

You do not need every extension point on day one.

SKILL A reusable Claude Code workflow. Start here.

AGENT A specialist Claude can delegate to for a focused job.

HOOK Automation that runs at a specific Claude Code event.

MCP A real connection to a system the plugin needs to read from or write to.

4 Build the smallest version

Use the standard plugin structure. Only `plugin.json` goes inside `.claude-plugin/`; the skill lives at the plugin root.

```
scaffold-kit/  
  .claude-plugin/plugin.json  
  skills/new-service/SKILL.md  
  README.md
```

`.claude-plugin/plugin.json`

```
{  
  "name": "scaffold-kit",  
  "description": "Scaffold a new service using Beef Co  
engineering conventions.",  
  "version": "0.1.0"  
}
```

`skills/new-service/SKILL.md`

```
---  
description: Scaffold a microservice using Beef Co engineering  
conventions.  
disable-model-invocation: true  
---
```

Create a new service named "\$ARGUMENTS".

Generate these files:

- `ci.yml`
- `Dockerfile`
- `README.md`
- `CODEOWNERS`

5 Finish the skill

Before writing files:

1. State the assumptions you are making.
2. List the files you plan to create.
3. Ask for confirmation if required by the current permissions.

Use the conventions already visible in this repo. Do not invent company standards.

Why this works: the skill is narrow, has a single input, names its output, and tells Claude when to stop and ask.

6 Run it, then prove it

Load the plugin

```
claude --plugin-dir ./scaffold-kit
```

Run the skill

```
/scaffold-kit:new-service payments-api
```

Validate before sharing

```
claude plugin validate
```

Done means:

- ☐ The README has the exact command.
- ☐ The skill runs from a fresh clone.
- ☐ Claude creates or proposes the files.
- ☐ The output is useful without extra explanation.

Add complexity later. Add agents, hooks, or MCP only after the one-skill version works and the missing piece is obvious.